Py-Bussilab Documentation

Contents

Module `bussilab` **3**

Install **4**
- Checking the installation . . . 4
- Setting autocompletion . . . 5

Getting started **5**
- Python . . . 5
- Examples . . . 5
- Command line . . . 5
- Changes . . . 5

Advanced stuff **5**
- Install with no dependencies . . . 6
- Building documentation . . . 6
- Implementing your modifications . . . 6
- Testing the code . . . 6
- Running and rendering jupyter examples from the command line . . . 7

Sub-modules . . . 7

Functions . . . 7
- Function `describe_submodule` . . . 7
- Function `import_submodules` . . . 8
- Function `list_submodules` . . . 8
- Function `required_conda` . . . 8
- Function `required_macports` . . . 8
- Function `required_pip` . . . 8

Module `bussilab.ann` **8**

Classes . . . 9
- Class `ANN` . . . 9
  - Methods . . . 9

Module `bussilab.cli` **10**

Tools to implement the command line interface **10**

Changelog . . . 11

Functions . . . 11
- Function `arg` . . . 11
- Function `cli` . . . 12
- Function `command` . . . 12
- Function `endgroup` . . . 13
- Function `group` . . . 13

Module `bussilab.cli_documentation` **13**

Documentation for command line tools **13**

list . . . 14

check . . . 14
wham . . . 14
jrun . . . 14
jremote . . . 15
pip_upgrade_all . . . 16
notify . . . 16
cron . . . 17
required . . . 18

Module `bussilab.clustering` 18
Changelog . . . 18
Functions . . . 19
Function `daura` . . . 19
Function `max_clique` . . . 20
Function `qt` . . . 20
Classes . . . 21
Class `ClusteringResult` . . . 21
Ancestors (in MRO) . . . 21
Instance variables . . . 21

Module `bussilab.coretools` 21
Changelog . . . 21
Functions . . . 22
Function `cd` . . . 22
Function `config` . . . 22
Function `config_path` . . . 22
Function `ensure_np_array` . . . 23
Function `import_numba_jit` . . . 23
Classes . . . 23
Class `Result` . . . 23
Examples . . . 23
Ancestors (in MRO) . . . 24
Descendants . . . 24
Class `TestCase` . . . 24
Ancestors (in MRO) . . . 24
Methods . . . 24

Module `bussilab.cron` 24
Changelog . . . 25
Functions . . . 25
Function `cron` . . . 25

Module `bussilab.installers` 26
Functions . . . 26
Function `build_viennarna` . . . 26
Function `install_viennarna` . . . 26

Module `bussilab.jremote` 27
Changelog . . . 27
Functions . . . 27
Function `find_free_port` . . . 27
Function `remote` . . . 27
Function `run_server` . . . 28

Module `bussilab.lohman` 28
Functions . . . 28
Function `lohman` . . . 28

Module `bussilab.maxent` 29

Functions . . . 29
Function maxent . . . 29
Classes . . . 30
Class MaxentResult . . . 30
Ancestors (in MRO) . . . 30
Instance variables . . . 30

Module bussilab.notify **31**
Changelog . . . 31
Functions . . . 32
Function notify . . . 32

Module bussilab.pip **34**
Changelog . . . 34
Functions . . . 34
Function install . . . 34
Function upgrade_all . . . 34
Function upgrade_self . . . 35

Module bussilab.potts **35**
Classes . . . 35
Class InferResult . . . 35
Ancestors (in MRO) . . . 35
Instance variables . . . 35
Class Model . . . 36
Methods . . . 36

Module bussilab.reports **37**
Changelog . . . 37
Functions . . . 37
Function workstations . . . 37

Module bussilab.rna2d **38**
Changelog . . . 38
Functions . . . 39
Function reset_default_parameters . . . 39
Function sample_to_numpy . . . 39
Function set_default_parameters . . . 40
Function set_random_seed . . . 40
Function suboptimal_to_numpy . . . 40
Classes . . . 40
Class Molecule . . . 40
Parameters . . . 41
Notes . . . 42
Instance variables . . . 42
Methods . . . 43

Module bussilab.wham **48**
Changelog . . . 48
Functions . . . 48
Function wham . . . 48
Classes . . . 50
Class WhamResult . . . 50
Ancestors (in MRO) . . . 51
Instance variables . . . 51

Module bussilab

Scientific and utility tools collected in one Python package.

Most features live in submodules, which are imported only when requested. Use list_submodules() to see what is available, or run bussilab list. Command usage is documented in cli_documentation[1]. Source code is on GitHub[2]. A PDF manual[3] is also available. This documentation describes version 0.0.52.

Install

This package supports Python 3.8 through 3.15. The recommended way to install this package depends on how you prefer to manage your python dependencies.

pip. If you manage your dependencies with pip and install packages in your home, use:

```
pip install --user bussilab
# if the user site is not on sys.path, add it to PYTHONPATH
export PYTHONPATH="$(python -c 'import site; print(site.USER_SITE)'):$PYTHONPATH"
# if the user scripts directory is not on PATH, add it
export PATH="$(python -c 'import site; print(site.USER_BASE + "/bin")'):$PATH"
```

Required packages will be downloaded and installed automatically in your home.

pip + venv. If you manage your dependencies with pip and work in a virtual environment[4], use:

```
pip install bussilab
```

Required packages will be downloaded and installed automatically in the virtual environment.

conda. If you manage your dependencies with conda, use:

```
conda install -c conda-forge -c bussilab py-bussilab
```

Required packages will be downloaded and installed automatically in the active conda environment.

macports. If you manage your dependencies with macports you might prefer to install required packages first. Since the list of requirements might change, it is recommended to use the bussilab package itself to obtain the list of requirements. You can do it as follows: [5]

```
# install pip and setuptools first
sudo port install py39-pip py39-setuptools
# install a bare version of the package, without dependencies
pip-3.9 install --user --no-deps bussilab
# if the user site is not on sys.path, add it to PYTHONPATH
export PYTHONPATH="$(python3.9 -c 'import site; print(site.USER_SITE)'):$PYTHONPATH"
# if the user scripts directory is not on PATH, add it
export PATH="$(python3.9 -c 'import site; print(site.USER_BASE + "/bin")'):$PATH"
# install the dependencies
sudo port install $(bussilab required --macports --pyver 39)
```

Notice that the list of required packages might change. It is thus recommended to run the commands above every time you update the bussilab package.

Checking the installation

Once the package is installed, check that Python can import it and its submodules:[6]

```
import bussilab
# this command can be used to check if all dependencies are in place:
bussilab.import_submodules()
```

[1] cli_documentation.html
[2] https://github.com/bussilab/py-bussilab
[3] ../bussilab.pdf
[4] https://docs.python.org/3/library/venv.html
[5] Notice that on macports the name of the python executable is python3.9 and the name of the pip installer is `pip-3.9`. A different python version should work as well.
[6] If you installed the package using the `--user` option, in order to be able to import the package you will have to make sure that the directory returned by the command `python -c 'import site; print(site.USER_SITE)'` is included in your python search path. If not, you can add it to the environment variable PYTHONPATH.

You can check the command-line entry point as well:[7]

```
bussilab -h
# this command can be used to check if all dependencies are in place:
bussilab check --import
```

Setting autocompletion

In order to benefit from autocompletion for the executable script you should install the argcomplete[8] package (with pip, conda, or macports) and add the following command to your .bashrc file:[9]

```
eval "$(register-python-argcomplete bussilab)"
```

Getting started

Python

The bussilab package itself contains lightweight infrastructure. Import submodules as needed:

```
from bussilab import wham
import numpy as np

bias = np.array([[1.0, 2.0], [2.0, 1.0]])
result = wham.wham(bias)
print(result.logW)
```

Submodules can also be accessed as attributes of bussilab; they are imported on first access. See each submodule's documentation for its Python API. Molecule additionally requires the optional ViennaRNA Python package.

Examples

In the examples[10] directory you can find a number of notebooks that can be used as a source of inspiration.

Command line

In addition, the bussilab script allows to access some functionality directly from the command line, without entering python. For example, to see the WHAM command's required arguments, run:

```
bussilab wham -h
```

See cli_documentation[11] for all subcommands.

Changes

Each submodule has its own release history. To receive updates when a release includes changes to a submodule you use, subscribe to its RSS feed. Feeds are available at `feeds/MODULE.xml` for modules with a changelog; for example, subscribe to the WHAM RSS feed[12] for WHAM updates.

Advanced stuff

Notice that instructions below assume you are using pip. If you use conda or macports you might have to adjust the commands.

[7] If you installed the package using the `--user` option, in order to be able to execute the script you will have to make sure that the directory returned by the command `python -c 'import site; print(site.USER_BASE + "/bin")'` is included in your PATH environment variable. Alternatively, if the bussilab script is not in your PATH you can run it as `python -m bussilab -h`.

[8] https://github.com/kislyuk/argcomplete

[9] If you are using macports, the command would be `eval "$(register-python-argcomplete-3.9 bussilab)"`.

[10] ../examples

[11] cli_documentation.html

[12] feeds/wham.xml

Install with no dependencies

You might want to ignore dependencies completely:

```
pip install --no-deps bussilab
```

You can still import the bussilab package and run the CLI, but submodules whose dependencies are missing may not import. Check them with:

```
bussilab check --import
```

This command reports imports that succeed or fail, then raises an error if any submodule fails to import.

Building documentation

You can build the documentation using pdoc3:

```
pip install pdoc3
pdoc3 -f --html -o doc/ bussilab
```

Documentation will be visible at `doc/bussilab/index.html`. This is normally not necessary, since the pre-built documentation of the latest version can be found at this link[13], but can be useful to test changes to the documentation before pushing them to GitHub.

Implementing your modifications

If you want to modify the Python source code you should download it from GitHub:

```
git clone https://github.com/bussilab/py-bussilab.git
cd py-bussilab
```

Since the module is written in pure python,[14] it can be used by just adding its path to PYTHONPATH and the bin directory to PATH:

```
export PATH="/path/to/py-bussilab/bin:$PATH"
export PYTHONPATH="/path/to/py-bussilab:$PYTHONPATH"
```

Testing the code

If you modified the code, it is recommended to test that your changes did not break existing features.

You can run the tests using pytest:

```
pip install pytest
pytest
```

Tests for optional features may be skipped when their dependencies are absent. Notice that the tests will import the bussilab module. Thus, if you want the version modified by you to be tested (and not another version that you might have installed with pip already) you should properly set the PYTHONPATH variable as explained above.

Static types can be checked using mypy:

```
pip install mypy
mypy bussilab
```

Run this optional check when changing type annotations.

Other static checks can be done with pyflakes:

```
pip install pyflakes
pyflakes bussilab
```

This check should succeed.

Additional static errors can be checked using pylint:

[13] https://bussilab.github.io/doc-py-bussilab
[14] This might change in the future.

```
pip install pylint
pylint -E bussilab
```

This check should succeed.

Running and rendering jupyter examples from the command line

The GitHub repository also contains jupyter examples. You can rerun all the jupyter examples from the command line:

```
cd examples
pip install jupyter jupyter_contrib_nbextensions matplotlib
./rerun.sh
```

You can render all the jupyter examples as html from the command line:

```
cd examples
pip install jupyter nbconvert
./render.sh
```

Sub-modules

- bussilab.ann
- bussilab.cli
- bussilab.cli_documentation
- bussilab.clustering
- bussilab.coretools
- bussilab.cron
- bussilab.installers
- bussilab.jremote
- bussilab.lohman
- bussilab.maxent
- bussilab.notify
- bussilab.pip
- bussilab.potts
- bussilab.reports
- bussilab.rna2d
- bussilab.wham

Functions

Function `describe_submodule`

```
def describe_submodule(
    module: str
) -> str
```

Return the first line of a submodule's source docstring without importing it.

Parameters

`module : str` Name of the module.

Returns

`str` The first line of the module docstring, or an empty string if it has no docstring. For `module=""`, return the first line of the package docstring.

Raises

`ModuleNotFoundError` If the module does not exist.

Examples

```
from bussilab import describe_submodule
print(describe_submodule("lohman"))
```

Function `import_submodules`

```
def import_submodules() -> None
```

Import all the available submodules, including hidden ones.

The main bussilab module does not explicitly import the available submodules so as not to slow down the behavior of the command line interface and to allow importing individual modules even if not all the dependencies of the other modules are installed. Use this function to import all the submodules (might take a few seconds).

Prints the result of each import. Import failures are collected and reported after all modules have been tried. This is useful for checking dependencies or preloading submodules.

Raises

Exception If one or more submodules raises ImportError during import. Other exceptions propagate immediately.

Function `list_submodules`

```
def list_submodules() -> List[str]
```

Return a list of all the available submodules.

The names come from Python files in the package directory. By default, modules whose names start with an underscore are omitted; `_hidden=True` includes them. Listing modules does not import them.

Returns

list A list of names of available submodules.

Examples

Print the available submodules and a short description for each of them.

```
from bussilab import list_submodules, describe_submodule
for m in list_submodules():
    print(m, describe_submodule(m))
```

Function `required_conda`

```
def required_conda() -> List[str]
```

Return package dependencies as a list for conda environments.

Function `required_macports`

```
def required_macports(
    pyver=''
) -> str
```

Return space-separated MacPorts dependencies.

pyver is the two-digit Python version suffix, such as `"39"`. When omitted, generic `py-` port names are returned.

Function `required_pip`

```
def required_pip() -> str
```

Return space-separated package dependencies for pip.

Module `bussilab.ann`

Module with artificial neural networks.

ANN can be constructed with `cuda=True`, in which case it will use cudamat.

Classes

Class `ANN`

```
class ANN(
    layers,
    random_weights=False,
    init_b=0.0,
    activation='softplus',
    cuda=False
)
```

Methods

Method `apply`

```
def apply(
    self,
    x
)
```

Method `applyVec`

```
def applyVec(
    self,
    x
)
```

Method `backward`

```
def backward(
    self,
    deriv,
    hidden
)
```

Method `backward_par`

```
def backward_par(
    self,
    deriv,
    hidden
)
```

Method `cuda_setup`

```
def cuda_setup(
    self
)
```

Method `deriv`

```
def deriv(
    self,
    x
)
```

Method `derivVec`

```
def derivVec(
    self,
    x
)
```

Method `derpar`

```
def derpar(
    self,
    x
)
```

Method `derparVec`

```
def derparVec(
    self,
    x
)
```

Method `dumpPlumed`

```
def dumpPlumed(
    self,
    path,
    style='ann',
    prefix=None,
    arguments=None
)
```

Method `forward`

```
def forward(
    self,
    x
)
```

Method `getpar`

```
def getpar(
    self
)
```

Method `setpar`

```
def setpar(
    self,
    par
)
```

Module `bussilab.cli`

Tools to implement the command line interface

This module defines the command-line parser and its subcommands. The same commands can be run from Python with cli():

```
from bussilab.cli import cli
cli("-h")
```

```
cli("wham -b bias") # provide the command line as a string
cli(["wham", "-b", "bias"]) # alternatively use a list
```

For command usage and options, see cli_documentation[15] or run `bussilab COMMAND -h`. The command(), arg(), group(), and endgroup() decorators register additional commands. Commands registered after this module is imported become available on the next cli() call.

The underlying modules also provide Python functions that work directly with data structures. For example, wham()(bias) accepts a NumPy array, while the wham command reads a file.

Changelog

- **v0.0.53**
  - Improve Jupyter discovery, URL handling, selection errors and SSH tunnel startup and cleanup; report Jupyter and browser launch failures.
  - Deprecated pip_upgrade_all in favor of rebuilding environments from pinned requirements; retained the command with a visible warning.
- **v0.0.52**
  - Added `notify --no-unfurl`, `--markdown-file`, and support for multiple paths with `--file`.
  - Made invalid command-line arguments exit with argparse's status code instead of printing a Python traceback; cli() still raises TypeError for parser errors by default when called from Python.
  - Preserved two-dimensional WHAM input from single-row and single-column files. Added clear errors for `--use-frame-weight` without a bias column and `--traj-weight` without values.
  - Made commands registered after import available to subsequent cli() calls by rebuilding the cached parser when needed.
  - Expanded command usage notes and clarified the generated CLI documentation.
- **v0.0.49**
  - Corrected numeric parsing of `wham --threshold` and the Jupyter `--port` options; fixed help and output for required and other command examples.
- **v0.0.42**
  - Added notification replies, reactions, quiet mode, footer control, and Screen log attachments with an optional line limit.
- **v0.0.41**
  - Added `jrun --lab` and improved Jupyter Screen socket names and usage notes.
  - Added cron quick-start options for resuming steps and event numbers, and allowed the period to come from the cron configuration.
- **v0.0.35**
  - Used shlex.split() for string arguments passed to cli(), preserving quoted values.
- **v0.0.34**
  - Added `cron --unique`, Jupyter Screen log support, and improved default Screen socket names.
- **v0.0.32**
  - Added file uploads to notify.
- **v0.0.12**
  - Added the required command for pip, conda, and MacPorts dependencies.
- **v0.0.2**
  - Introduced the command registration helpers, Python cli() entry point, generated command documentation, and the initial subcommands.

Functions

Function `arg`

```
def arg(
    *name,
    **kwargs
)
```

[15] cli_documentation.html

Decorator that adds an argument to a command line tool.

Parameters are passed to the parser.add_argument() function. Write it **below** the command() decorator.

Function `cli`

```
def cli(
    arguments: str | List[str] = '',
    *,
    prog: str | None = '',
    use_argcomplete: bool = False,
    throws_on_parser_errors: bool = True
) -> int | None
```

Executes a command line tool from python.

Run a registered subcommand from Python using the same arguments as the bussilab executable. A string is split with shlex.split(); a list preserves each argument exactly.

Parameters

`arguments` : str or list Command line arguments. If a string is passed, it is first split using shlex.split()

`prog` : str or None Name used in help and error messages. The default uses an empty name; None uses sys.argv[0]. **Mostly for internal use**.

`use_argcomplete` : bool If True, the autocomplete function of argcomplete module is called on the parser, so as to allow autocompletion in the command line tool. If argcomplete module is not installed, nothing is done and no failure is reported. **Mostly for internal use**.

`throws_on_parser_errors` : bool If True, convert nonzero parser exits to TypeError. If False, return the parser's exit code. The executable uses False so invalid arguments exit with argparse's status code. **Mostly for internal use**.

Returns

None or int Returns None after a successful command or a help request. With `throws_on_parser_errors=False`, returns the parser's nonzero exit code for invalid arguments. Exceptions raised by a command propagate.

Function `command`

```
def command(
    name: str,
    help: str | None = None,
    description: str | None = None,
    **kwargs
)
```

Decorator that registers a function as a subcommand.

Write this decorator **above** the other decorators arg(), group(), and endgroup(). Newly registered commands are available on the next cli() call.

Parameters

`name` : str Name of the subcommand (will be used on the command line)

`help` : str Short help message for the subcommand (one line).

`description` : str, optional Longer description. If not provided, it is set to a copy of help.

`kwargs` Other parameters are passed as is to the add_parser function of argparse.

Examples

Simple command line tool that accepts a single `--out` argument followed by a string and call the function do_something with that string as an argument.

```
from bussilab.cli import command, arg

@command("subcommand")
```

```
@arg("--out")
def myfunc(out):
    do_something(out)
```

Function endgroup

```
def endgroup(
    f: Callable | None = None
)
```

Decorator that ends a group of arguments for a command line tool.

See group().

Function group

```
def group(
    title: str | Callable | None = None,
    description: str | None = None,
    exclusive: bool | None = None,
    required: bool | None = None
)
```

Decorator that adds a group of arguments for a command line tool.

Write it **below** the command() decorator. It should be followed by a number of arg() decorators and by a closing endgroup() decorator.

Parameters

title : str The name of the group. Can only be used for non exclusive groups.
description : str A description of the group. Can only be used for non exclusive groups.
exclusive : bool If True, the arguments belonging to this group are mutually exclusive
required : bool If True, one of the arguments at least should be passed. Can only be used for exclusive groups.

Examples

This is a simple command line tool that accepts three arguments (`-a`, `-b`, or `-c`), mutually exclusive. When ran, it will just print booleans showing if these arguments were passed.

```
from bussilab.cli import command, group, arg, endgroup

@command("doit")
@group(exclusive=True)
@arg("-a", action='store_true')
@arg("-b", action='store_true')
@arg("-c", action='store_true')
@endgroup
def check(a, b, c):
   print(a, b, c)
```

Module `bussilab.cli_documentation`

Documentation for command line tools

This page is generated from the registered commands when this module is imported. Each section includes the corresponding `bussilab COMMAND -h` output, followed by any longer usage notes from the command's docstring. For calling commands from Python or registering new commands, see `bussilab.cli`[16].

[16] cli.html

list

```
usage: bussilab list [-h]

List available python modules.

options:
  -h, --help  show this help message and exit
```

Print the available Python submodules and their short descriptions.

check

```
usage: bussilab check [-h] [--import]

Check installed features

options:
  -h, --help  show this help message and exit
  --import    check if all the submodules can be imported
```

Import every submodule with `--import` to check installed dependencies.

wham

```
usage: bussilab wham [-h] -b BIAS [-o OUT] [--use-frame-weight]
                     [--traj-weight TRAJ_WEIGHT [TRAJ_WEIGHT ...]] [-T T]
                     [-m MAXITER] [-t THRESHOLD] [-v]

Perform binless WHAM

options:
  -h, --help            show this help message and exit
  -b, --bias BIAS       File containing bias potential
  -o, --out OUT         Output file with log weights (default: None)
  --use-frame-weight    read frame weights from the last bias-file column
                        (default: False)
  --traj-weight TRAJ_WEIGHT [TRAJ_WEIGHT ...]
                        relative weights of the bias columns (default: None)
  -T, --temperature T   system temperature in energy units (default: 1.0)
  -m, --maxiter MAXITER
                        maximum number of iterations (default: 1000)
  -t, --threshold THRESHOLD
                        threshold for convergence (default: 1e-40)
  -v, --verbose
```

Read a whitespace-delimited bias matrix and run binless WHAM.

Each row is a frame and each column is a Hamiltonian's bias potential. With `--use-frame-weight`, the last column supplies frame weights and at least one preceding bias column is required. Print log weights to standard output, or write them to `--out` with four decimal places. Iteration count, error, and log partition functions are always printed to standard output. See bussilab.wham for the Python API.

jrun

```
usage: bussilab jrun [-h] [-d] [--lab] [--port PORT] [--screen-cmd SCREEN_CMD]
                     [--screen-log SCREEN_LOG] [--python-exec PYTHON_EXEC]
                     [-S SOCKNAME] [--no-screen] [--keep-ld-library-path]
                     [--detach]

Run jupyter server
```

```
options:
  -h, --help            show this help message and exit
  -d, --dry-run         show command instead of executing it (default: False)
  --lab                 use jupyterlab (default: False)
  --port PORT           set port (default: 0)
  --screen-cmd SCREEN_CMD
                        screen command (default: screen)
  --screen-log SCREEN_LOG
                        screen logfile (no logfile by default) (default: )
  --python-exec PYTHON_EXEC
                        python executable (default: )
  -S, --sockname SOCKNAME
                        screen sockname (default: (path):(port):jupyter)
  --no-screen           do not run screen (default: False)
  --keep-ld-library-path
                        (ignored, this is the default now) (default: False)
  --detach              detach screen (default: False)
```

This is a tool to run a jupyter server within a screen command. The typical usage would be

```
cd /path/to/your/notebook/dir
bussilab jrun
```

A free port is identified first (can be overridden with the `--port` option) and a jupyter server is then run inside a screen instance. You will thus have to type `CTRL+aCTRL+d` in order to detach the screen letting it run in the background.

Alternatively, you can immediately detach the screen with

```
cd /path/to/your/notebook/dir
bussilab jrun --detach
```

Notice that, since the server is run inside a screen instance, in order to visualize python outputs that has been sent directly to the terminal you should connect to the screen instance later. By default, a socket name containing the path where the server is running is used, with `/` replaced by `:`. It should thus be easy to use `screen -ls` to find the proper screen instance.

jremote

```
usage: bussilab jremote [-h] [-d] [-l] [--port PORT] [-i INDEX]
                        [--python-exec PYTHON_EXEC] [--server-url SERVER_URL]
                        [--open-cmd OPEN_CMD]
                        server

Run jupyter client

positional arguments:
  server                server URL (e.g. giorgione.phys.sissa.it)

options:
  -h, --help            show this help message and exit
  -d, --dry-run         show command instead of executing it (default: False)
  -l, --list-only       only report a list of servers (default: False)
  --port PORT           set port (default: 0)
  -i, --index INDEX     choose server, by default interactive choice (default:
                        0)
  --python-exec PYTHON_EXEC
                        remote python executable (e.g. module load python3
                        python-home; python) (default: python)
  --server-url SERVER_URL
```

```
                        URL on server (default: choose interactively)
                        (default: )
  --open-cmd OPEN_CMD   browser command and options (detected automatically by
                        default) (default: )
```

This is a tool to connect to a remote running jupyter server. The typical usage would be

```
bussilab jremote server.url
```

A list of jupyter servers running on the selected machines will be shown, and one of them can be picked typing its progressive number. In case there is a single server running, it will be opened by default.

The browser opens after the local SSH listener is ready. Keep this terminal open while using Jupyter; press `CTRL+c` to close the tunnel. Use `--server-url` to supply a loopback HTTP(S) URL directly, including its path and any authentication token. `--open-cmd` accepts a browser command with options. A dry run can still use SSH to discover servers; combine `--dry-run --list-only` to avoid making a connection.

Notice that if the name of the python executable on the server is different from plain python you can override it with `--python-exec`. You can also run other scripts before, for instance loading relevant modules:

```
bussilab jremote giorgione.phys.sissa.it            --python-exec "module load python3 python-home;
```

If you recurrently connect to the same workstation, it is convenient to write a small script like this one, call it jremote and put it in your path:

```
export PYTHONPATH=/path/to/bussilab/source
python -m bussilab jremote giorgione.phys.sissa.it --python-exec "module load python3 python-home ;
```

Replace python with the name of your Python interpreter if needed.

pip_upgrade_all

```
usage: bussilab pip_upgrade_all [-h] [--user]

Deprecated: upgrade all packages in place with pip

options:
  -h, --help  show this help message and exit
  --user      install/upgrade in user location (default: False)
```

Deprecated: upgrade all discovered packages in place with pip.

Build and validate a new environment from pinned requirements instead, keeping the previous environment available for rollback. This command remains available for compatibility. It does not preserve version pins and assumes all discovered packages are managed by pip.

Legacy usage:

```
bussilab pip_upgrade_all
```

If you installed packages in your home you should use

```
bussilab pip_upgrade_all --user
```

notify

```
usage: bussilab notify [-h] [-m MESSAGE] [--markdown-file MARKDOWN_FILE]
                       [-c CHANNEL | -u UPDATE | -d DELETE | -r REPLY |
                       -R REPLY_BROADCAST | -X REACT] [-f FILE [FILE ...]]
                       [-t TITLE] [--no-footer] [--no-unfurl]
                       [--screenlog SCREENLOG]
                       [--screenlog-maxlines SCREENLOG_MAXLINES] [--type TYPE]
                       [--token TOKEN] [-q]
```

```
Send a notification to Slack

options:
  -h, --help            show this help message and exit
  -m, --message MESSAGE
                        message (default: None)
  --markdown-file MARKDOWN_FILE
                        path to a standard Markdown message (default: None)
  -c, --channel CHANNEL
                        channel (check ~/.bussilabrc by default) (default:
                        None)
  -u, --update UPDATE   url of the message to be updated (default: None)
  -d, --delete DELETE   url of the message to be deleted (default: None)
  -r, --reply REPLY     url of the message to be replied (default: None)
  -R, --reply-broadcast REPLY_BROADCAST
                        url of the message to be broadcast-replied (default:
                        None)
  -X, --react REACT     react to a message (default: None)
  -f, --file FILE [FILE ...]
                        path(s) to files to be uploaded (incompatible with -u
                        and -d) (default: None)
  -t, --title TITLE     title of the message (default: None)
  --no-footer           ignore footer (default: False)
  --no-unfurl           disable link and media previews (default: False)
  --screenlog SCREENLOG
                        screenlog file (default: None)
  --screenlog-maxlines SCREENLOG_MAXLINES
                        maximum number of lines in screenlog (0 means all)
                        (default: 0)
  --type TYPE           'plain_text' or 'mrkdwn' (use --markdown-file for
                        standard Markdown) (default: mrkdwn)
  --token TOKEN         token (check ~/.bussilabrc by default) (default: None)
  -q, --quiet           quiet (do not write output) (default: False)
```

This is a tool to send a notification to Slack. See the documentation of bussilab.notify.

cron

```
usage: bussilab cron [-h] [--quick-start]
                     [--quick-start-skip-steps QUICK_START_SKIP_STEPS]
                     [--quick-start-event QUICK_START_EVENT]
                     [--cron-file CRON_FILE] [--screen-cmd SCREEN_CMD]
                     [--screen-log SCREEN_LOG] [--no-screen]
                     [--keep-ld-library-path] [-S SOCKNAME]
                     [--python-exec PYTHON_EXEC] [--detach] [--unique]
                     [--window] [--period PERIOD] [--max-times MAX_TIMES]

Run cron

options:
  -h, --help            show this help message and exit
  --quick-start         run immediately (default: False)
  --quick-start-skip-steps QUICK_START_SKIP_STEPS
                        skip steps on quick start (default: None)
  --quick-start-event QUICK_START_EVENT
                        event number for quick start (default: None)
  --cron-file CRON_FILE
                        path to cron file (default: None)
  --screen-cmd SCREEN_CMD
```

```
                        screen command (default: screen)
  --screen-log SCREEN_LOG
                        screen logfile (no logfile by default) (default: )
  --no-screen           do not run screen (default: False)
  --keep-ld-library-path
                        (ignored, this is the default now) (default: False)
  -S, --sockname SOCKNAME
                        screen sockname (default: (path):cron)
  --python-exec PYTHON_EXEC
                        python executable (default: )
  --detach              detach screen (default: False)
  --unique              allow only one screen with this socket (default:
                        False)
  --window              run a new window within the same screen (default:
                        False)
  --period PERIOD       period (seconds) default read from cron file or set to
                        3600 (default: None)
  --max-times MAX_TIMES
                        maximum number of calls (default: None)
```

Run scheduled steps from a YAML cron configuration.

Use `--cron-file` to select a file; otherwise the package configuration is used. The cron section can be a list of steps or a mapping with steps and an optional period in seconds. By default, the command starts in a GNU Screen session; `--no-screen` runs it in the current terminal. `--quick-start` runs the first iteration immediately, and `--max-times` limits iterations.

required

```
usage: bussilab required [-h] [--macports | --conda] [--pyver PYVER]

print requirements

options:
  -h, --help     show this help message and exit
  --macports     macports syntax (default: False)
  --conda        conda syntax (default: False)
  --pyver PYVER  pyversion (e.g. 38), for macports only (default: )
```

Print package requirements for pip, conda, or MacPorts.

Module `bussilab.clustering`

Cluster frames from an adjacency or distance matrix.

max_clique() and daura() use a Boolean adjacency matrix. The former repeatedly extracts a maximum-weight clique; the latter repeatedly selects the frame with the most neighbors and removes its neighborhood. qt() uses a distance matrix and builds clusters whose pairwise distances do not exceed a cutoff. All three methods can use nonnegative frame weights and return a ClusteringResult with cluster members and their total weights.

Dense input matrices require memory proportional to the square of the number of frames. daura() also accepts sparse adjacency matrices, whose storage scales with the number of neighboring frame pairs. max_clique() can take exponential time on difficult graphs.

Changelog

- **v0.0.52**
  - Added sparse adjacency and neighbor-index iterable input to daura(); both use CSR neighbor rows and a priority queue as frames are removed.

  - Made daura() and qt() reuse the full input matrix instead of repeatedly copying smaller matrices; updated cluster degrees incrementally and reduced temporary arrays. The input matrices are left unchanged.
  - Fixed handling of absent adjacency self-edges, zero frame weights, floating-point degree cancellation, and `max_clusters=0`. qt() now includes distances exactly equal to the cutoff.
  - Against commit 4dec1e4, daura() was about 17x faster without weights and 21x faster with weights on a 1,000-frame singleton matrix. qt() was about 1.5x faster without weights and 1.4x faster with weights on 1,200 random 2D points at cutoff 0.12 (96 and 98 clusters, respectively). These are medians of three warm runs on the same machine, using float64 distances and, in the weighted cases, float64 weights.
- **v0.0.40**
  - In qt(), prioritized smaller-diameter clusters when cluster sizes tie and higher-weight frames when candidates are equally distant.
- **v0.0.39**
  - Fixed weighted clustering in daura() and qt().

Functions

Function `daura`

```
def daura(
    adj,
    weights=None,
    *,
    min_size=0,
    max_clusters=None
)
```

Cluster by repeatedly removing the largest adjacency neighborhood.

This is the algorithm introduced in Daura et al, Angew. Chemie (1999). The input is a symmetric adjacency matrix; self-edges are optional.

Parameters

`adj` : array_like, scipy sparse matrix, or iterable of iterables A dense matrix has adj[i,j] equal to 1 (or True) when frames i and j are adjacent, and 0 otherwise. Alternatively, each inner iterable can contain the integer indexes of one frame's neighbors. Sparse matrices and neighbor iterables are converted to CSR internally. Only positive sparse entries represent neighbors. Self-edges are optional. Adjacency must be symmetric, so include both directions of each neighbor pair. A square, symmetric, binary Python list is interpreted as a dense matrix; use iter(rows) to pass it as neighbor rows. NumPy arrays are always treated as dense matrices.

`weights` : array_like, optional weights[i] contains the finite, nonnegative weight of the i-th frame.

`min_size` : number Minimum cluster size. Clusters smaller than this size are not reported. When using weights, the cluster size is defined as the sum of the weights of the members of the cluster.

`max_clusters` : int Maximum number of clusters; zero returns an empty result.

Example

```
from bussilab import clustering
import scipy.spatial.distance as distance
dist=distance.squareform(distance.pdist(trajectory))
clustering.daura(dist<0.7)

##### sparse_adj can be a scipy.sparse CSR, CSC, or COO matrix.
clustering.daura(sparse_adj,weights=frame_weights)

##### One iterable of neighbor indexes per frame also works.
clustering.daura([[1],[0,2],[1]])
```

Function `max_clique`

```
def max_clique(
    adj,
    weights=None,
    *,
    min_size=0,
    max_clusters=None,
    use_networkit=False
)
```

Clustering algorithm used in Reisser et al, NAR (2020)[17].

Parameters

`adj` : array_like, square matrix adj[i,j] contains 1 (or True) if frames i and j are adjacent, 0 (or False) otherwise.

`weights` : array_like, optional weights[i] contains the finite, nonnegative weight of the i-th frame.

`min_size` : number Minimum cluster size. Clusters smaller than this size are not reported. When using weights, the cluster size is defined as the sum of the weights of the members of the cluster.

`max_clusters` : int Maximum number of clusters; zero returns an empty result.

`use_networkit` : bool, optional if True, use a networkit implementation that seems to be faster. It requires python package networkit to be installed in advance!

Example

```
from bussilab import clustering
import scipy.spatial.distance as distance
dist=distance.squareform(distance.pdist(trajectory))
clustering.max_clique(dist<0.7)
```

Function `qt`

```
def qt(
    distances,
    cutoff,
    weights=None,
    *,
    min_size=0,
    max_clusters=None
)
```

Quality threshold clustering.

The method is explained in the original paper[18]. The implementation has been adapted from this one[19], which is also released under a GPL licence. Thus, if you use this algorithm please cite this article[20], which also discusses the important differences between this algorithm and the Daura et al algorithm in the context of analysing molecular dynamics simulations. Additionally, mention which exact version of the bussilab package you used.

This implementation supports weights and arbitrary distance metrics. Singleton clusters are included unless excluded by min_size or max_clusters. When candidate clusters have the same total weight, the one with smaller diameter is selected. When growing a candidate cluster, equally distant frames are considered by descending weight. A float32 distance matrix uses less memory than float64, although rounding can change results near the cutoff or a tie.

Parameters

`distances` : array_like, square matrix distances[i,j] contains the distance between frames i and j. The matrix should be symmetric; its diagonal is ignored.

[17] https://doi.org/10.1093/nar/gkz1184
[18] https://doi.org/10.1101/gr.9.11.1106
[19] https://github.com/rglez/QT
[20] https://doi.org/10.1021/acs.jcim.9b00558

cutoff : number Maximum pairwise distance within a cluster (inclusive).
weights : array_like, optional weights[i] contains the finite, nonnegative weight of the i-th frame.
min_size : number Minimum cluster size. Clusters smaller than this size are not reported. When using weights, the cluster size is defined as the sum of the weights of the members of the cluster.
max_clusters : int Maximum number of clusters; zero returns an empty result.

Example

```
from bussilab import clustering
import numpy as np
import scipy.spatial.distance as distance
dist=distance.squareform(distance.pdist(trajectory))
clustering.qt(dist,0.7)

clustering.qt(np.array(dist,dtype='float32'),0.7)
```

Classes

Class `ClusteringResult`

```
class ClusteringResult(
    *,
    method: str,
    clusters: list,
    weights: list | None
)
```

Clusters and their sizes, in the order selected by a clustering method.

Ancestors (in MRO)

- bussilab.coretools.Result
- builtins.dict

Instance variables

Variable `clusters` List of clusters, each containing original frame indices.

Variable `method` Name of the method: max_clique(), daura(), or qt().

Variable `weights` Total frame weight of each cluster, or its size if unweighted.

Module `bussilab.coretools`

General purpose tools.

Changelog

- **v0.0.52**
  - Removed the unused file_or_path() helper.
  - Fixed TestCase.assertEqualFile() to close the first file when opening the second file fails.
  - Made config_path() use Path.home() for the default configuration path, allowing home directory lookup when HOME is unset.
  - Documented configuration loading and fixed the Result constructor example.
- **v0.0.45**
  - Changed import_numba_jit() to return numba.njit instead of numba.jit.
- **v0.0.41**
  - Added import_numba_jit(), with a warning and an undecorated Python fallback when importing Numba fails.

- **v0.0.38**
  - Made `cd(create=True)` create missing parent directories as well.
- **v0.0.37**
  - Added the create option to cd().
- **v0.0.35**
  - Fixed missing-file error messages in TestCase.assertEqualFile() for path objects.
- **v0.0.34**
  - Added config_path() to expose the configuration file path.
- **v0.0.33**
  - Added explicit missing-file failure messages to TestCase.assertEqualFile().
- **v0.0.29**
  - Made ensure_np_array() convert NumPy scalars to zero-dimensional arrays and corrected its return annotation to include None.
- **v0.0.18**
  - Made Result.___dir___() return keys in alphabetical order.
- **v0.0.7**
  - Introduced array conversion, file opening, result containers, temporary directory changes, and file-comparison assertions.
  - Added YAML configuration loading from `~/.bussilabrc` or an explicit path.

Functions

Function `cd`

```
def cd(
    newdir: os.PathLike,
    *,
    create: bool = False
)
```

Context manager to temporarily change working directory.

Can be used to change working directory temporarily making sure that at the end of the context the working directory is restored. Notably, it also works if an exception is raised within the context.

Parameters

`newdir` : path Path to the desired directory.
`create` : bool (default False) Create directory first. If the directory exists already, no error is reported

Examples

```
from bussilab.coretools import cd
with cd("/path/to/dir"):
    do_something() # this is executed in the /path/to/dir directory
do_something_else() # this is executed in the original directory
```

Function `config`

```
def config(
    path: os.PathLike | None = None
)
```

Read YAML configuration from a path, defaulting to `~/.bussilabrc`.

Scalar values are returned as strings, including numbers and booleans. An empty file returns None. File and YAML parsing errors propagate to the caller.

Function `config_path`

```
def config_path(
    path: os.PathLike | None = None
```

```
)
```

Return the supplied path, or `~/.bussilabrc` when omitted.

Function `ensure_np_array`

```
def ensure_np_array(
    arg
) -> numpy.ndarray | None
```

Convert arg to np.array if necessary.

Function `import_numba_jit`

```
def import_numba_jit()
```

Return a numba.njit object. If import fails, return a fake jit object and emits a warning.

Currently, the returned object can only be used as @njit (no option). It might be extended to allow more jit options.

Classes

Class `Result`

```
class Result(
    *args,
    **kwargs
)
```

Base class for objects returning results.

It allows one to create a return type that is similar to those created by scipy.optimize.minimize. The string representation of such an object contains a list of attributes and values and is easy to visualize on notebooks.

Examples The simplest usage is this one:

```
from bussilab import coretools

class MytoolResult(coretools.Result):
    """Result of a mytool calculation."""
    pass

def mytool():
    a = 3
    b = "ciao"
    return MytoolResult(a=a, b=b)

m=mytool()
print(m)
```

Notice that the class variables are dynamic: any keyword argument provided in the class constructor will be processed. If you want to enforce the class attributes you should add an explicit constructor. This will also allow you to add pdoc docstrings. The recommended usage is thus:

```
from bussilab import coretools

class MytoolResult(coretools.Result):
    """Result of a mytool calculation."""
    def __init__(self, a, b):
        super().__init__()
        self.a = a
        """Documentation for attribute a."""
```

```
        self.b = b
        """Documentation for attribute b."""

def mytool():
    a = 3
    b = "ciao"
    return MytoolResult(a=a, b=b)

m = mytool()
print(m)
```

Ancestors (in MRO)

- builtins.dict

Descendants

- bussilab.clustering.ClusteringResult
- bussilab.maxent.MaxentResult
- bussilab.potts.InferResult
- bussilab.wham.WhamResult

Class `TestCase`

```
class TestCase(
    methodName='runTest'
)
```

Improved base class for test cases.

Extends the unittest.TestCase class with some additional assertion.

Create an instance of the class that will use the named test method when executed. Raises a ValueError if the instance does not have a method with the specified name.

Ancestors (in MRO)

- unittest.case.TestCase

Methods

Method `assertEqualFile`

```
def assertEqualFile(
    self,
    file1: os.PathLike,
    file2: os.PathLike | None = None
)
```

Check if two files are equal.

Parameters

`file1` : path Path to the first file
`file2` : path, optional Path to the second file. If not provided, defaults to `file1+".ref"`.

Module `bussilab.cron`

Run independent Python and Bash steps periodically, optionally inside Screen.

A running scheduler reloads its YAML configuration at scheduled events. Read or validation failures are logged and the last valid configuration stays active; startup requires a valid configuration. A changed period reschedules the next event. Individual step errors, nonzero exits and timeouts are logged with

the event and step index, and remaining steps still run. Child output is inherited by cron, including its Screen log when enabled. Detached Screen jobs retain their existing lifecycle.

The explicit selfupdate step upgrades bussilab and may update dependencies in the running environment before restarting. Use it only for environments deliberately maintained in place. For shared or version-pinned environments, build and validate a new environment, then restart the scheduler there.

Changelog

- **v0.0.53**
  - Retain the last valid configuration after reload failures, with explicit diagnostics; validate configuration structure and positive periods.
  - Continue independent steps after errors, nonzero exits and timeouts, logging the failing step and traceback.
  - Fix fractional-second scheduling arithmetic while retaining local-week alignment and existing skip/delay expressions.
- **v0.0.42**
  - Use integer event/period division when evaluating skip/delay schedules.
- **v0.0.41**
  - Read the period from configuration when not explicitly overridden, and reload it during operation. Support a mapping with period and steps alongside the original list form; rename jobs to steps.
  - Resume at the next step after reboot/self-update, preserving the scheduled event and remaining iteration count. Add quick-start resume options.
  - Account for fractional seconds when waiting for scheduled events; improve event/step logging and continue after unsupported Screen reboot requests.
  - Shorten long Screen socket names with a hash suffix.
- **v0.0.35**
  - Add self-update and reboot steps using replacement Screen windows, preserving detached mode and checking for Screen 4.1 or newer.
  - Align scheduling to the local week and allow expressions in skip/delay.
  - Parse Screen/Python command strings with shlex.split() and respect iteration limits before waiting for another event.
- **v0.0.34**
  - Add Bash steps, skip/delay scheduling and unique Screen session checks.
  - Derive default socket names from configuration paths and preserve LD_LIBRARY_PATH by default when launching through Screen.
- **v0.0.33**
  - Add optional LD_LIBRARY_PATH forwarding to Screen sessions.
- **v0.0.11**
  - Add max_times to limit scheduled iterations and introduce cron tests.
- **v0.0.2**
  - Introduce periodic Python scripts from YAML configuration, optional Screen sessions, quick start, subprocess timeouts and configuration reloads.

Functions

Function `cron`

```
def cron(
    *,
    quick_start: bool = False,
    quick_start_skip_steps: int = 0,
    quick_start_event: int = 0,
    cron_file: str = '',
    screen_cmd: str = 'screen',
    screen_log: str = '',
    no_screen: bool = True,
    keep_ld_library_path: bool = True,
```

```
    sockname: str = '(path):cron',
    python_exec: str = '',
    detach: bool = False,
    period: int | None = None,
    max_times: int | None = None,
    unique: bool = False,
    window: bool = False
)
```

Module `bussilab.installers`

Install and build selected external scientific packages.

At present, the functions in this module support Google Colab only.

Functions

Function `build_viennarna`

```
def build_viennarna(
    version: str = '2.7.2-patch2',
    output_directory: str | os.PathLike | None = None,
    jobs: int | None = None,
    verbose: bool = True
) -> Tuple[pathlib.Path, pathlib.Path]
```

Build a patched ViennaRNA archive in Google Colab.

The archive and its complete build log receive the same basename.

Parameters

`version` Patched release identifier, for example `"2.7.2-patch2"`.
`output_directory` Destination for the .tgz and .log files. The default is the current working directory.
`jobs` Number of parallel make jobs. The default is at most two, to avoid excessive memory use while compiling the SWIG wrapper.
`verbose` Print progress information and the final elapsed time.

Returns

archive, log Paths of the generated archive and build log.

Function `install_viennarna`

```
def install_viennarna(
    version: str = '2.7.2-patch2',
    prefix: str | os.PathLike | None = None,
    force: bool = False,
    verbose: bool = True
)
```

Install a precompiled patched ViennaRNA distribution in Google Colab.

Parameters

`version` Patched release identifier, for example `"2.7.2-patch2"`.
`prefix` Installation directory. The default is ViennaRNA below the current working directory.
`force` Download and extract the archive again if the installation directory already exists.
`verbose` Print progress information.

Returns

module The imported :mod:RNA module.

Raises

RuntimeError If the function is not running in Google Colab, the archive is malformed, or the installed module cannot be imported.

Module `bussilab.jremote`

Module implementing tools for remote jupyter connections.

This module contains some utilities that are mostly designed to be used as command line tools. The interface of the functions defined in this module is subject to changes. One should instead use the subcommands jrun and jremote in the cli_documentation[21] submodule.

Discovery uses the remote Python environment selected by `--python-exec`. Connections are forwarded through a local loopback SSH listener; keep the terminal open while using Jupyter. Discovery supports modern Jupyter Server and older notebook/Lab installations.

Changelog

- **v0.0.53**
  - Use JSON server discovery with legacy fallbacks and deduplicate results. Missing Lab installations no longer prevent notebook discovery.
  - Accept loopback HTTP(S) URLs with custom paths, queries and authentication settings. Validate server selection and handle empty results and EOF.
  - Wait for the SSH listener before opening the browser, report forwarding failures, and terminate and reap the tunnel on exit.
  - Report server and browser launch failures; support browser command options.
- **v0.0.42**
  - Handle the JupyterLab 3.6 server-list output format.
- **v0.0.41**
  - Add JupyterLab launching and discovery alongside notebook servers.
  - Shorten long Screen socket names using a hash suffix.
- **v0.0.38**
  - Improve handling of invalid interactive server selections.
- **v0.0.36**
  - Add an explicit server URL option to bypass server discovery.
- **v0.0.35**
  - Parse Python and Screen command options with shlex.split().
- **v0.0.34**
  - Add Screen logging and preserve LD_LIBRARY_PATH by default.
- **v0.0.32**
  - Add optional LD_LIBRARY_PATH forwarding to Screen sessions.
- **v0.0.2**
  - Introduce Jupyter launching in Screen and remote connections through SSH.

Functions

Function `find_free_port`

```
def find_free_port()
```

Returns the number of a free port.

Function `remote`

```
def remote(
    server: str,
    python_exec: str = 'python',
    dry_run: bool = False,
    list_only: bool = False,
    server_url: str = '',
```

[21] cli_documentation.html

```
    port: int = 0,
    index: int = 0,
    open_cmd: str = ''
)
```

Discover remote Jupyter servers and open one through an SSH tunnel.

python_exec is a remote shell command and can include module-loading commands. Dry runs can perform discovery over SSH, but do not open a browser or start a tunnel. Combine with list_only to avoid SSH entirely.

Function `run_server`

```
def run_server(
    dry_run: bool = False,
    port: int = 0,
    screen_cmd: str = 'screen',
    screen_log: str = '',
    no_screen: bool = False,
    keep_ld_library_path: bool = True,
    python_exec: str = '',
    sockname: str = '(path):(port):jupyter',
    lab: bool = False,
    detach: bool = False
)
```

Runs a jupyter server inside a screen command.

This function is only designed to be used as a command line tool.

Module `bussilab.lohman`

Module implementing Lohman model for helicases.

Functions

Function `lohman`

```
def lohman(
    t,
    ku: float,
    kd: float,
    n: int = 1,
    boundaries: Tuple[float, float] = (0.0, 1.0)
) -> float | numpy.ndarray
```

Lohman model for helicases.

Compute the fraction of unwound helices as a function of time. See Lucius et al, Biophys J 2003[22].

Parameters

`t` : float or sequence or np.ndarray Time. If a sequence or np.ndarray is provided, the function is computed for all values and an array is returned.
`ku` : float Unwinding rate.
`kd` : float Dissociation rate.
`n` : int, optional Step size
`boundaries` : tuple with 2 elements Result is mapped to this range.

Returns

[22] https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1303449/

float or np.ndarray The fraction of unfolded helices at a given time t. If an array is provided for t, an array is returned containing the fractions at all the times. If boundaries is provided, the fraction is linearly mapped into the boundaries[0], boundaries[1] range.

Module `bussilab.maxent`

Tools to perform reweighting using MaxEnt.

Functions

Function `maxent`

```
def maxent(
    traj,
    reference,
    *,
    logW=None,
    maxiter: int = 1000,
    verbose: bool = False,
    lambdas=None,
    l2=None,
    l1=None,
    method: str = 'L-BFGS-B',
    regularization: Callable | None = None,
    tol: float | None = None,
    options=None,
    cuda=False
)
```

Tool that computes new weights to enforce reference values.

This tools process a an array containing the observables computed along a trajectory and returns new weights that satisfy the maximum entropy principle and so that weighted averages agree with reference values.

Parameters

`traj` : array_like A 2D array (lists or tuples are internally converted to numpy arrays). traj[i,j] is j-th observable computed in the i-th frame. If traj is a CUDAMatrix object, then cudamat is used irrespectively of the bool parameter cuda.

`reference` : array_like A 1D array (lists or tuples are internally converted to numpy arrays) containing the reference values to be enforced. If the i-th element is a tuple or an array with 2 elements, they are interpreted as boundaries. For instance, `reference=[1.0,(2.0,3.0)]` will make sure the first observable has value 1 and the second observable is within the range (2,3). Boundaries equal to `+np.inf` or `-np.inf` can be used to imply no boundary. Notice that boundaries in the form (A,B) where both A and B are finite are implemented by adding fictitious variables in a way that is transparent to the user. Boundaries in the form (A,B) where one of A or B is finite and the other is infinite are implemented as boundaries on lambdas. Boundaries in the form (A,A) are interpreted as constraints.

`logW` : array_like A 1D array (lists or tuples are internally converted to numpy arrays) containing the logarithm of the a priori weight of the provided frames.

`lamdbas` : array_like A 1D array with initial values of lambda. A good guess will minimize faster. A typical case would be recycling the lambdas obtained with slighlty different regularization parameters.

`l2` : None, float, or array_like Prefactor for L2 regularization. If None, no regularization is applied. If float, the same factor is used on all the lambdas. If it is an array, it should have length equal to len(reference).

`l1` : None, float, or array_like Prefactor for L1 regularization. If None, no regularization is applied. If float, the same factor is used on all the lambdas. If it is an array, it should have length equal to len(reference).

`regularization` : callable or None A function that takes as argument the current lambdas and return an tuple containing the regularization function and its derivatives. For instance, passing a function defined as `def reg(x): return (0.0001*0.5*np.sum(x**2),0.0001*x)` is equivalent to passing `l2=0.0001`.

`verbose` : bool If True, progress informations are written on stdout.

`method` : str Minimization method. See documentation of scipy.optimize.minimize.

`maxiter` : int Maximum number of iterations

`tol` : float or None Tolerance for minimization. See documentation of scipy.optimize.minimize.

`options` : dict Arbitrary options passed to scipy.optimize.minimize.

`cuda` : bool or None (default False) Use cuda. If None, chosen based on the availability of the cudamat library.

Notes On Using Cuda

Note that for normal datasets the cost of transfering the traj object to the GPU dominates. It it however possible to transfer the traj object first to the GPU with `cu_traj=cm.CUDAMatrix(traj)` and then reuse it for multiple calls (e.g. for a hyper parameter scan).

Classes

Class `MaxentResult`

```
class MaxentResult(
    *,
    logW_ME: numpy.ndarray,
    lambdas: numpy.ndarray,
    averages: numpy.ndarray,
    gamma: float,
    success: bool,
    message: str,
    nfev: int,
    nit: int
)
```

Result of a maxent() calculation.

Ancestors (in MRO)

- bussilab.coretools.Result
- builtins.dict

Instance variables

Variable `averages` np.ndarray with len(reference) elements, resulting averages.

Variable `gamma` float containing the resulting likelihood Gamma.

Variable `lambdas` np.ndarray with len(reference) elements, optimized Lagrangian multipliers.

Variable `logW_ME` np.ndarray with traj.shape[0] elements, logarithms of the optimized weights.

Variable `message` str reporting the possible reason of failuer of the minimizer.

Variable `nfev` int reporting the number of function evaluations.

Variable `nit` int reporting the number of iterations in the minimization procedure.

Variable `success` bool reporting the success of the minimizer.

Module `bussilab.notify`

Module implementing Slack notifications.

This module sends notifications through an App installed in the Slack workspace.

Changelog

- **v0.0.52**
  - Added standard Markdown messages from files and options to disable link and media previews.
  - Added uploading multiple files in one call. File titles now use the supplied title or each file's basename, and threaded uploads include their message correctly.
  - Validated Slack message, file, and reaction URLs before the corresponding operation, including checks that each operation receives the right URL type.
  - Separated rate-limit retries from transient-error retries, bounded total wait time, and improved handling of `Retry-After` and transport errors.
  - Increased the Screen log limit by using native Markdown blocks and fixed several file-upload edge cases.
- **v0.0.49**
  - Delayed the first file-share lookup to allow Slack time to complete an upload.
- **v0.0.48**
  - Added jitter and stronger backoff to Slack API and file-share retries.
- **v0.0.46**
  - Switched file uploads to Slack's files_upload_v2 when available, including uploads in reply threads, and fixed returned file links.
- **v0.0.45**
  - Improved delays when retrying Slack rate limits and server errors.
- **v0.0.42**
  - Added replies, broadcast replies, reactions, reaction deletion, Screen log attachments, and checks for incompatible operations.
  - Added bounded retries for rate limits and server errors and limits for long messages and titles.
- **v0.0.32**
  - Added uploading and deleting Slack files.
- **v0.0.18**
  - Added compatibility with Slack SDK 3 while retaining Slack client 2.
- **v0.0.2**
  - Introduced Slack notifications with configuration-based authentication, message updates and deletion, and optional footers.

Some settings are needed first for authentication. It is recommended to add a file named .bussilabrc to your home directory with the following content:

```
notify:
  token: xoxb-00000
  channel: U00000
```

The token here should be provided by the administrator of your workspace. The channel should be the Slack ID associated to your user. It can be found looking in your Slack profile. With these settings, the tool will send notifications to you by default.

Notifications can then be sent using either the command line:

```
bussilab notify --message "text here"
```

or from python:

```
from bussilab.notify import notify
notify("text here")
```

Notice that the message is optional. Even with an empty message, the footer will allow you to reconstruct from which machine and directory the message was sent from. This might be sufficient for your goal.

Link and media previews can be disabled using the unfurl option:

```
bussilab notify --message "https://example.com" --no-unfurl
```

or from python:

```
notify("https://example.com", unfurl=False)
```

A file written in standard Markdown can be sent without adding a title or footer:

```
bussilab notify --markdown-file report.md
```

or from python:

```
notify(markdown_file="report.md")
```

You can also indicate a specific channel for the notification using the channel option:

```
bussilab notify --message "text here" --channel "project-myproject"
```

or from python:

```
from bussilab.notify import notify
notify("text here", channel="project-myproject")
```

This will only work if the App has been added to the specified channel.

The following syntax can be used to upload one or more files:

```
bussilab notify --message "text here" --file /path/to/file
bussilab notify --message "text here" --file first.dat second.dat
```

or from python:

```
from bussilab.notify import notify
notify("text here",file="/path/to/file")
notify("text here",file=["first.dat", "second.dat"])
```

The commands above will return the URL of the message. This URL can be used later to update or delete them or to post reactions:

```
url=$(bussilab notify --message "text here")
bussilab notify --update $url --message "revised message"
bussilab notify --react $url:heart

# this will remove only the reaction:
bussilab notify --delete $url:heart

# this will remove the entire message:
bussilab notify --delete $url

url=$(bussilab notify --message "text here")
```

or from python:

```
from bussilab.notify import notify
url=notify("text here")
notify("revised message", update=url)
notify(react=url+":heart")
notify(delete=url+":heart")
notify(delete=url)
```

In these cases, the channel is not needed and should not be provided. Notice that you will only be able to update or delete messages sent through the App.

Functions

Function `notify`

```
    def notify(
        message: str = '',
```

```
        channel: str = None,
        *,
        markdown_file: str = '',
        react: str = None,
        update: str = None,
        delete: str = None,
        reply: str = None,
        reply_broadcast: str = None,
        title: str = '',
        screenlog: str = '',
        screenlog_maxlines: int = 0,
        footer: bool = True,
        unfurl: bool = True,
        type: str = 'mrkdwn',
        file: str | Sequence[str] = '',
        token: str = None
    )
```

Tool to send notifications to Slack.

Parameters

`message` : str A string that will form the body of the message.
`channel` : None or str The channel. By default, taken from your `~/.bussilabrc` configuration file.
`markdown_file` : str Read a standard Markdown message from this UTF-8 text file. This implies `type="markdown"` and cannot be combined with a non-empty message argument.
`update` : None or str The URL of a message to be updated.
`delete` : None or str The URL of a message to be deleted. By passing a URL concatenated with the string `":name_of_reaction"` you can delete a reaction. By passing comma-separated URLs you can delete a message and all the files shared with it.
`reply` : None or str The URL of a message to be replied
`reply_broadcast` : None or str The URL of a message to be broadcast-replied
`react` : None or str The URL of a message to which you want to add a reaction, followed by the string `:name_of_the_reaction`
`file` : str or sequence of str The path of a file to be uploaded, or paths of multiple files.
`title` : str The title of the notification.
`screenlog` : str The path of a GNU Screen log file. Its contents are displayed in a fenced native Markdown block.
`screenlog_maxlines` : int If positive, include only this many lines from the end of the Screen log.
`footer` : bool If True, a footer is added with current user, machine, and directory.
`unfurl` : bool If False, link and media previews are disabled when posting a message or a reply. The option does not affect file-upload comments or message updates.
`type` : str The type of message. Can be “mrkdwn”, “plain_text”, or “markdown”. Standard Markdown uses a native Markdown block; title, screenlog, file uploads, and footers are not supported.
`token` : None or str The token. By default, taken from your `~/.bussilabrc` configuration file.

Returns

```
str
    A string with the URL of the sent message.
    In case the <code>delete</code> keyword is used, it returns an empty
    string.
    When files are uploaded, it returns comma-separated URLs for
    the message followed by every file.
```

Example

```
from bussilab.notify import notify
notify("send this message")
```

See bussilab.notify for more examples.

Module `bussilab.pip`

Helpers for invoking pip with the current Python interpreter.

For shared or version-pinned environments, build and validate a new environment and publish it under a new version or date, keeping the previous one available for rollback. Upgrading an existing environment does not preserve its pins.

Changelog

- **v0.0.53**
  - Deprecated upgrade_all() and the pip_upgrade_all command in favor of rebuilding environments from pinned requirements.
  - Replaced pkg_resources package discovery with importlib.metadata.
- **v0.0.35**
  - Added upgrade_self() for upgrading bussilab, used by cron's self-update step. Added optional subprocess timeouts to all pip helpers.
- **v0.0.2**
  - Introduced install() and upgrade_all(), invoking pip with the current Python interpreter and supporting upgrades and user installations.

Functions

Function `install`

```
def install(
    packages: str | List[str],
    *,
    upgrade: bool = False,
    user: bool = False,
    timeout: int | None = None
)
```

Install one or more packages with pip.

Install packages making sure they get installed with the currently used python interpreter.

Parameters

`packages` : str or list Package to be installed/upgraded. If a list is passed, multiple packages are installed/upgraded.
`upgrade` : bool, optional if True, run pip with `--upgrade`.
`user` : bool, optional if True, run pip with `--user`.

Function `upgrade_all`

```
def upgrade_all(
    user: bool = False,
    *,
    timeout: int | None = None
)
```

Upgrade all discovered packages using pip (deprecated).

This modifies the environment in place, without preserving version pins or providing validation or rollback. Build a new environment from pinned requirements instead. This function remains available for compatibility. It assumes all discovered packages are managed by pip; with `user=True`, packages visible outside the user location can also be installed there.

Parameters

`user` : bool, optional if True, install/upgrade packages with the `--user` option.

Function `upgrade_self`

```
def upgrade_self(
    *,
    user: bool = False,
    timeout: int | None = None
)
```

Upgrade bussilab in place using pip, potentially updating dependencies.

Intended for environments deliberately maintained in place, including cron's explicit selfupdate step. For shared or pinned environments, publish a new environment instead.

Module `bussilab.potts`

Module containing a tool to solve Potts models by enumeration

See Model.

Classes

Class `InferResult`

```
class InferResult(
    *,
    h: numpy.ndarray,
    J: numpy.ndarray,
    averages: numpy.ndarray,
    loglike: float,
    success: bool,
    message: str,
    nfev: int,
    nit: int
)
```

Result of a Model.infer() calculation.

Ancestors (in MRO)

- bussilab.coretools.Result
- builtins.dict

Instance variables

Variable `J` np.ndarray, optimized h.

Variable `averages` np.ndarray, resulting averages.

Variable `h` np.ndarray, optimized h.

Variable `loglike` float containing the resulting likelihood Gamma.

Variable `message` str reporting the possible reason of failuer of the minimizer.

Variable `nfev` int reporting the number of function evaluations.

Variable `nit` int reporting the number of iterations in the minimization procedure.

Variable `success` bool reporting the success of the minimizer.

Class `Model`

```
class Model(
    size: int,
    colors: int = 1,
    shifted: bool = False,
    fullmatrix: bool = True
)
```

Init model. size: number of spins colors: number of colors fullmatrix: set to False to use less memory (slower)

Methods

Method `compute`

```
def compute(
    self,
    h: numpy.ndarray,
    J: numpy.ndarray
)
```

Compute averages <sigma_i,sigma_j> for a coupling matrix J. Returns (a,b) with a=free energy and b=averages

Method `draw`

```
def draw(
    self,
    h: numpy.ndarray,
    J: numpy.ndarray,
    n: int
)
```

Compute averages for a coupling matrix J sampling n states.

Method `fixJ`

```
def fixJ(
    self,
    J: numpy.ndarray
)
```

Method `infer`

```
def infer(
    self,
    averages: numpy.ndarray,
    nseq: int = 1,
    reg: Callable | None = None
)
```

Method `loglike`

```
def loglike(
    self,
    h: numpy.ndarray,
    J: numpy.ndarray,
    ave: numpy.ndarray
)
```

Compute -log likelihood for a coupling matrix J with averages ave. Returns (a,b) with a=-log likelihood and b=derivatives

Method `random_couplings`

```
def random_couplings(
    self,
    seed: int | None = None
)
```

Module `bussilab.reports`

Workstation status reports collected over SSH.

:func:workstations() returns a compact Markdown table for posting with `bussilab.notify.notify(report, type="markdown")`. Workstations can be passed explicitly or read from the workstations() configuration list. CPU and GPU utilization and disk occupancy are shown as percentages, with connection status circles and warning symbols. Set `short=False` to append raw output.

Changelog

- **v0.0.53**
  - Replaced emoji-based status summaries with a Markdown table containing Workstation, CPU, GPU, Disk and Temp columns. Added numeric percentages, green/red connection indicators and warning symbols beside affected values. Raw output now follows the table in fenced code blocks.
  - Detect `nvidia-smi` in the remote shell's PATH automatically, ignoring the legacy nvidia setting. Query GPU utilization as CSV; missing tools omit GPU usage, while failed queries retain CPU/disk results and show unavailable GPU utilization.
  - Handle CPU fields without intervening spaces and decimal-comma values; request the C locale remotely. Quote disk paths and handle spaces in mount points. Distinguish connection failures from parsing errors.
  - Added automated tests using mocked SSH calls and local fake utilities; no SSH connection is required by the tests.
- **v0.0.41**
  - Increased the SSH timeout from 20 to 60 seconds.
- **v0.0.34**
  - Added a random output marker to discard login banners before parsing.
- **v0.0.33**
  - Default workstation names to their SSH address and initialize the temp path for string entries. Include percentages in disk-space warnings.
- **v0.0.31**
  - Added configurable temp-disk monitoring, defaulting to `/var`, with a warning above 70% occupancy.
- **v0.0.2**
  - Introduced SSH workstation reports with CPU, NVIDIA GPU and disk status, configuration-based host lists, timeouts and optional raw output.

Functions

Function `workstations`

```
def workstations(
    wks: List | None = None,
    short: bool = True
)
```

Return a Markdown table of workstation CPU, GPU and disk usage.

Send the result with `notify(message=report, type="markdown")`. A green circle means SSH returned data; a red circle marks a connection failure or timeout. Collection errors appear in the affected metric cells. CPU usage is 100 minus idle; GPU percentages follow device order. Warning values carry a warning symbol: CPU id+us+ni below 80%, disk usage at least 90%, temp usage above 70%, and unavailable GPU utilization. Disk and Temp show the used percentage of their configured paths. An em dash means no GPU was detected. With `short=False`, raw output follows the table in fenced code blocks.

The legacy nvidia workstation setting is ignored. Machines without `nvidia-smi` omit GPU status; a failed GPU query produces a warning.

Module `bussilab.rna2d`

RNA secondary-structure ensembles built on ViennaRNA.

The module wraps RNA.fold_compound with a higher-level model that is useful when the energy depends on whether particular nucleotides are paired. Compared with using ViennaRNA directly, :class:Molecule provides continuous per-nucleotide and sparse exact-base-pair penalties without losing sub-centikcal precision, arbitrary coupled biases on selected paired/unpaired states, and transparent combination of the resulting conditioned ensembles. The same interface then provides MFE structures, structure evaluation, partition functions, base-pair probabilities, ensemble diversity, exact free-energy derivatives, centroid and MEA structures, positional entropy, ensemble defect, Boltzmann sampling, suboptimal ensembles, and joint pairing probabilities.

The wrapper also manages thermodynamic parameter sets, caches calculations, recovers from partition-function scaling failures, and defaults to disabling ViennaRNA's partition-function energy smoothing so that partition functions, sampling, MFE, evaluation, and suboptimal structures share one energy model. The :func:sample_to_numpy() and :func:suboptimal_to_numpy() helpers convert structure collections to NumPy pair tables and normalized log weights.

ViennaRNA's Python package is an optional dependency of bussilab but is required when constructing a :class:Molecule.

Changelog

- **v0.0.53**
  - Fixed suboptimal enumeration and coverage with soft constraints on unpatched ViennaRNA by using base-pair constraints in the fallback.
- **v0.0.52**
  - Added exact state mixtures with state_positions and state_biases, allowing coupled paired/unpaired-state energy biases. Multiple disjoint state sets and reduce_state_space are supported; ensemble results combine all state components.
  - Added Molecule hard constraints (force_paired, force_unpaired).
  - Added centroid(), mea(), positional_entropy(), and ensemble_defect() for the complete ensemble, including state mixtures.
  - evaluate() and ensemble_defect() accept zero-based NumPy pair tables; evaluate() also accepts a two-dimensional batch of tables.
  - Added sparse exact-pair base_pair_biases and d_free_energy_d_base_pair_biases(); continuous biases retain their sub-0.01 kcal/mol precision with unpatched ViennaRNA. Impossible pairs may remain in the bias list and have zero ensemble probability.
  - Added evaluate() for one structure or a one-dimensional iterable of structures or (structure, value) pairs, including output from sample() and suboptimal_structures(). Structures can be rescored by another Molecule with the same sequence. Noncanonical or too-short pairs, and structures violating hard constraints, return infinite energy.
  - Added sample_to_numpy() and suboptimal_to_numpy() for zero-based pair tables and normalized log weights, plus mean_bp_distance() and free-energy derivatives with respect to one-body and state biases.
  - Exposed the resolved temperature and Boltzmann constant as read-only Molecule.temperature and Molecule.boltzmann_constant properties, along with sequence, no_lonely_pair, pf_smooth, NaCl, and parameters. Renamed the constructor's seq argument to sequence; seq remains a deprecated keyword alias.

  - Added Molecule.to_kwargs() to inspect and reuse resolved constructor settings.
  - Removed the samples and suboptimal_delta options from pairing_correlation_matrix(); it now always returns the exact partition-function result.
  - Changed sample() to return unweighted structure strings by default; `sample(weights=True)` retains structures with importance log weights.
  - Added set_random_seed() for reproducible sampling without changing NumPy's global random state.
  - Added no_lonely_pair and pf_smooth options and set_default_parameters() / reset_default_parameters() for new molecules. Partition-function smoothing now defaults to off so ensemble and structure energies use the same model.
  - Fixed suboptimal enumeration with positive one-body penalties on unpatched ViennaRNA and parameter-set loading with ViennaRNA 2.7.2.
  - Fixed validation of temperature and salt settings: zero or non-finite temperatures and non-finite salt concentrations are now rejected.
- **v0.0.51**
  - Introduced Molecule with continuous per-nucleotide pairing penalties, MFE prediction, partition functions, base-pair probabilities, suboptimal enumeration, sampling, and suboptimal coverage.
  - Added pairing_correlation_matrix() for joint paired-state probabilities from partition functions, samples, or suboptimal structures.

Functions

Function `reset_default_parameters`

```
def reset_default_parameters()
```

Restore the original thermodynamic defaults for new molecules.

Function `sample_to_numpy`

```
def sample_to_numpy(
    samples,
    *,
    deduplicate=True
)
```

Convert sampled dot-bracket structures to NumPy arrays.

The input may be either a sequence of structure strings, as returned by `Molecule.sample(weights=False)`, or a sequence of `(structure, log_weight)<code> pairs, as returned by </code>Molecule.sample(weights=True` Mixed inputs are rejected.

Structures are represented by zero-based pair tables: states[k, i] is the index paired with nucleotide i in structure k, or `-1` when it is unpaired. This is a NumPy-oriented adaptation of ViennaRNA's pair table format. A row table can be converted to ViennaRNA's convention with `np.concatenate(([len(table)], table + 1))`: ViennaRNA prepends the sequence length, uses one-based partner indices, and uses zero for an unpaired nucleotide.

Parameters

`samples` : sequence of str or sequence of (str, float) Unweighted structures or structures with unnormalized log weights.

`deduplicate` : bool, default=True If True, merge identical structures. Unweighted occurrences are combined through their counts; supplied log weights are combined using logarithmic addition, then order structures by decreasing combined weight. Equal-weight structures retain first-occurrence order. If False, retain every occurrence in input order.

Returns

`states` : ndarray of int16, shape (n_structures, sequence_length) Zero-based pair tables, with `-1` denoting an unpaired nucleotide.

logweights : ndarray of float, shape (n_structures,) Normalized log weights, satisfying `sum(exp(logweights)) == 1` up to floating-point precision.

Function `set_default_parameters`

```
def set_default_parameters(
    *,
    temperature=None,
    no_lonely_pair=None,
    pf_smooth=None,
    NaCl=None,
    parameters=None
)
```

Update the default thermodynamic parameters for new molecules.

Parameters set to None are left unchanged. Existing molecules are not affected. Use :func:reset_default_parameters() to restore all original defaults, including ViennaRNA's default salt concentration.

Function `set_random_seed`

```
def set_random_seed(
    seed
)
```

Seed ViennaRNA and the private NumPy RNG used by :meth:Molecule.sample().

The seed affects subsequent sampling by this module and direct ViennaRNA sampling in the current process. It does not change NumPy's global random state. ViennaRNA's RNG is process-global, so interleaved ViennaRNA calls can affect reproducibility.

Parameters

seed : int Unsigned 32-bit integer seed.

Function `suboptimal_to_numpy`

```
def suboptimal_to_numpy(
    suboptimal,
    temperature
)
```

Convert suboptimal dot-bracket structures and energies to NumPy arrays.

Structures use the same zero-based pair-table representation documented by :func:sample_to_numpy(). Energies are converted to Boltzmann log weights at the explicitly supplied temperature.

Parameters

suboptimal : sequence of (str, float) Structure and energy pairs, in kcal/mol, as returned by Molecule.suboptimal_structures().
temperature : float Temperature in kelvin.

Returns

states : ndarray of int16, shape (n_structures, sequence_length) Zero-based pair tables, with `-1` denoting an unpaired nucleotide.
logweights : ndarray of float, shape (n_structures,) Normalized Boltzmann log weights, satisfying `sum(exp(logweights)) == 1` up to floating-point precision.

Classes

Class `Molecule`

```
class Molecule(
    sequence: str = None,
```

```
    *,
    seq=None,
    lambdas1d=None,
    base_pair_biases=None,
    temperature=None,
    force_paired=None,
    force_unpaired=None,
    state_positions=None,
    state_biases=None,
    reduce_state_space=True,
    no_lonely_pair=None,
    pf_smooth=None,
    NaCl=None,
    parameters=None
)
```

Model an RNA secondary-structure ensemble with pairing-dependent energies.

Molecule retains ViennaRNA's nearest-neighbor secondary-structure energy and augments it with continuous one-body pairing penalties and, optionally, arbitrary many-body biases on selected paired/unpaired states. For a structure s, the modeled energy is

```
E(s) = E_ViennaRNA(s) + sum_i lambda_i paired_i(s) + sum_ij b_ij pair_ij(s) + state_bias(s),
```

where paired_i is one when nucleotide i is paired and zero otherwise. Positive lambda_i values disfavor pairing and negative values favor it. pair_ij is one only when nucleotides i and j form the exact base pair, and b_ij is its sparse base-pair bias. A state-bias tensor can express cooperative or non-additive interactions between any chosen nucleotides; multiple tensors contribute additively.

Internally, pairing penalties are mapped to ViennaRNA soft constraints. State biases are represented exactly as a mixture of dynamic-programming ensembles hard-conditioned on selected nucleotides. Public results are combined across those components, so callers use the same API for an ordinary ensemble and for a biased state mixture.

Calculations are performed lazily and cached. The class supports MFE prediction, evaluation of a supplied dot-bracket structure, ensemble free energies, base-pair probabilities and diversity, centroid and MEA structures, positional entropy and ensemble defect, derivatives with respect to penalties and state biases, Boltzmann sampling, suboptimal enumeration and coverage, and joint nucleotide-pairing probabilities. Structure energies and state biases are expressed in kcal/mol; temperatures are supplied in kelvin. The sequence and resolved model settings are available as read-only properties.

Parameters

`sequence : str` RNA sequence. The former seq keyword is accepted with a deprecation warning.

`lambdas1d : array-like, optional` Per-nucleotide pairing penalties (kcal/mol). Positive values penalize pairing, whereas negative values favor pairing. If omitted, all penalties are zero.

`base_pair_biases : iterable of (int, int, float), optional` Sparse exact-base-pair biases (i, j, energy) in kcal/mol, using zero-based nucleotide indices. A contribution is applied only when nucleotides i and j pair with each other. Positive energies disfavor the pair and negative energies favor it. Pair orientation is normalized internally; duplicate and non-finite entries are rejected. An impossible pair is allowed but has no effect on the ensemble, and its free-energy derivative is zero. Evaluating a structure containing an impossible pair returns infinite energy. The iterable order is retained for :meth:d_free_energy_d_base_pair_biases.

`force_paired : array-like of int, optional` Zero-based indices of nucleotides that are required to be paired, without specifying their pairing partners.

`force_unpaired : array-like of int, optional` Zero-based indices of nucleotides that are required to be unpaired.

`state_positions : array-like of int or sequence of array-like, optional` Zero-based indices defining binary paired/unpaired states. A flat sequence defines one state set. A sequence of sequences defines multiple disjoint state sets whose energy biases are additive. If state_biases is omitted, every state is assigned zero bias.

state_biases : array-like or sequence of array-like, optional Energy biases (kcal/mol) for the states defined by state_positions. For one state set, its shape must be `(2,) * len(state_positions)`. For multiple sets, provide one tensor per set, with shape `(2,) * len(positions)`. Index zero denotes an unpaired nucleotide and index one a paired nucleotide. It cannot be provided without state_positions.

reduce_state_space : bool, default=True If True, represent the final selected state position as an equivalent 1D pairing penalty, reducing the number of dynamic-programming ensembles from 2**N to 2**(N-1). If False, use one hard-conditioned ensemble for every state.

no_lonely_pair : bool or None, default=None If True, use ViennaRNA's noLP model option. MFE and suboptimal enumeration exclude isolated base pairs, but ViennaRNA's partition function and sampling can still include some structures containing them. If None, use the current module default.

pf_smooth : bool or None, default=None Whether ViennaRNA should smooth energies in partition-function Boltzmann factors. If None, use the current module default, which is initially False. If True, partition functions and sampling use ViennaRNA's smoothed model, while MFE, evaluation, and suboptimal structures continue to use its unsmoothed energy model.

temperature : float or None, default=None Temperature in kelvin. If None, use the current module default, which is initially 310.15 K.

NaCl : float or None, default=None Sodium concentration (M). If None, use the current module default, which initially selects ViennaRNA's default value.

parameters : {"turner1999", "turner2004", "andronescu2007", "langdon2018"} or None Thermodynamic parameter set. If None, use the current module default.

Notes A state set containing N positions normally requires `2**N` conditioned dynamic-programming ensembles. By default, the final position of each set is represented by an equivalent constant shift and one-body penalty, reducing the total from `2**sum(N_k)` to `2**(sum(N_k) - K)` for K non-empty state sets. Consequently, large numbers of state positions can still be expensive even with `reduce_state_space=True`.

Standard ViennaRNA builds round soft-constraint energies to the nearest 0.01 kcal/mol. This class corrects that rounding: partition functions use a residual callback, sampling uses rejection or importance reweighting, and MFE and suboptimal candidates are widened and rescored. Therefore the requested continuous one-body and exact-base-pair penalties are retained even without a patched ViennaRNA build that supports them natively.

For comparing the partition function with a Boltzmann sum of evaluated structures, set both `no_lonely_pair=False` and `pf_smooth=False` explicitly (module defaults can be changed). With `no_lonely_pair=True`, ViennaRNA's partition function and sampling may retain lonely-pair structures that MFE and suboptimal enumeration exclude. With `pf_smooth=True`, the partition function and sampling use smoothed Boltzmann factors, while MFE, evaluation, and suboptimal enumeration remain unsmoothed. Either switch can make suboptimal_coverage inconsistent with the partition-function energy convention.

Hard constraints specify only whether a nucleotide is paired; they do not specify its partner. evaluate assigns infinite energy to structures that violate them. As in ViennaRNA, centroid and mea summarize marginal pairing probabilities without enforcing force_paired on their output. A returned structure may therefore violate that constraint when a required nucleotide's probability is spread over multiple partners. If no valid structure exists, mfe raises RuntimeError. Structures are pseudoknot-free, as in ViennaRNA's standard single-strand folding grammar.

Instance variables

Variable `NaCl` Sodium concentration in M, or None for ViennaRNA's default.

Variable `boltzmann_constant` Boltzmann constant used by this module, in kcal/(mol K).

Variable `no_lonely_pair` Whether ViennaRNA's no-lonely-pair model option is enabled.

Variable `parameters` Thermodynamic parameter-set name used by this molecule.

Variable `pf_smooth` Whether ViennaRNA smooths partition-function energies.

Variable `sequence` RNA sequence used by this molecule.

Variable `temperature` Temperature used by this molecule, in kelvin.

Methods

Method `base_pairing_probability`

```
def base_pairing_probability(
    self
)
```

Return the base-pairing probability matrix.

Returns

ndarray Symmetric NxN matrix whose element (i,j) is the equilibrium probability that nucleotides i and j form a base pair.

Method `centroid`

```
def centroid(
    self
)
```

Return the centroid structure and its mean base-pair distance.

The centroid contains every pair whose probability in the complete ensemble exceeds 0.5. Like ViennaRNA's centroid, it is a summary of pairing probabilities and can violate force_paired when no one partner of a required nucleotide has probability above 0.5.

Returns

(str, float) Dot-bracket structure and expected base-pair distance from a random structure in the ensemble.

Method `d_free_energy_d_base_pair_biases`

```
def d_free_energy_d_base_pair_biases(
    self
)
```

Return free-energy derivatives for the requested base-pair biases.

For an entry (i, j, energy) in base_pair_biases, the corresponding derivative is the equilibrium probability that the exact base pair (i, j) is formed. Values are returned in the same order as the constructor entries. An impossible pair has derivative zero.

Returns

ndarray One-dimensional array containing one exact-base-pair probability per base_pair_biases entry.

Method `d_free_energy_d_lambdas1d`

```
def d_free_energy_d_lambdas1d(
    self
)
```

Return the derivatives of the free energy with respect to lambdas1d.

The derivative for nucleotide i is its equilibrium pairing probability,

`dF / d lambda_i = <s_i>`.

Returns

ndarray One-dimensional array containing one derivative per nucleotide.

Method `d_free_energy_d_state_biases`

```
def d_free_energy_d_state_biases(
    self
)
```

Return the derivatives of the free energy with respect to state biases.

The derivative with respect to a state's energy bias is the equilibrium probability of that state.

Returns

ndarray or list of ndarray With the flat, single-set state_positions syntax, an array with the same shape as state_biases. With the list-of-sets syntax, one array per set, including when the list contains only one set.

Raises

ValueError If this molecule was not initialized with state biases.

Method `ensemble_defect`

```
def ensemble_defect(
    self,
    structure
)
```

Return the expected fraction of incorrectly paired nucleotides.

A nucleotide is correct when it has the same partner as in the target structure, or is unpaired in both. The expectation uses the complete ensemble, including state mixtures. The target is a dot-bracket structure of the same length as the sequence; it need not satisfy the molecule's hard constraints or contain only canonical pairs.

Parameters

`structure` : str or ndarray Target structure in dot-bracket notation or as a zero-based NumPy pair table with `-1` for unpaired nucleotides.

Returns

float Ensemble defect divided by sequence length, in [0, 1].

Method `evaluate`

```
def evaluate(
    self,
    structure
)
```

Return exact free energies for one or more secondary structures.

Parameters

`structure` : str, ndarray, or iterable A dot-bracket structure, a zero-based NumPy pair table with `-1` for unpaired nucleotides, or an iterable of structures. A two-dimensional integer array supplies a batch of pair tables. Iterable elements may also be (structure, value) pairs, whose values are ignored. This accepts the results of :meth:suboptimal_structures, sample(), `sample(weights=True)`, :func:sample_to_numpy(), and :func:suboptimal_to_numpy(). Structures generated by one Molecule may be rescored under another molecule's biases when the sequences match.

Returns

float or ndarray Exact free energy in kcal/mol, including continuous pairing penalties, exact-base-pair biases, and state biases. Scalar input returns a float; batched input returns an array with shape (n_structures,). Noncanonical pairs and pairs shorter than the minimum loop length, or structures violating hard constraints, return infinity. Malformed structures raise ValueError.

Method `mea`

```
def mea(
    self,
    gamma=1.0
)
```

Return a maximum expected accuracy structure and its score.

The score is `2 * gamma * sum(p_ij)` over selected pairs plus the unpaired probabilities at unpaired positions. ViennaRNA's MEA optimizer receives the complete ensemble's base-pair probabilities, including all state-mixture components. As with ViennaRNA's default MEA, the returned summary structure need not satisfy force_paired.

Parameters

`gamma` : float, default=1.0 Nonnegative relative weight of correctly predicted base pairs.

Returns

(str, float) Dot-bracket structure and maximum expected accuracy score.

Method `mean_bp_distance`

```
def mean_bp_distance(
    self
)
```

Return the mean base-pair distance between two ensemble structures.

The diversity is `sum_ij p_ij * (1 - p_ij)`, using the symmetric base-pairing probability matrix. For a state mixture, p_ij is the probability in the combined ensemble.

Returns

float Expected base-pair distance between two independent structures.

Method `mfe`

```
def mfe(
    self
)
```

Return the minimum-free-energy structure.

The returned energy always corresponds to the exact continuous pairing penalties, even when ViennaRNA internally rounds soft constraints.

Returns

`structure` : str Dot-bracket representation of the MFE structure.
`energy` : float Exact free energy (kcal/mol).

Method `pairing_correlation_matrix`

```
def pairing_correlation_matrix(
    self
)
```

Return the joint pairing-probability matrix.

Element (i, j) is the probability that nucleotides i and j are simultaneously paired, irrespective of their pairing partners.

The matrix is computed exactly using constrained partition-function calculations.

Returns

ndarray Symmetric NxN matrix whose diagonal contains the pairing probabilities.

Method `positional_entropy`

```
def positional_entropy(
    self
)
```

Return the partner-choice entropy of each nucleotide in bits.

For nucleotide i, the outcomes are pairing with each possible partner j and remaining unpaired. The probabilities come from the complete ensemble, including all state-mixture components. Thus the result includes uncertainty between components as well as within them; it is not the entropy of the full structure distribution.

Returns

ndarray One entropy value per nucleotide.

Method `sample`

```
def sample(
    self,
    number,
    weights=False
)
```

Generate Boltzmann-distributed secondary structures.

By default, rejection sampling corrects ViennaRNA's rounded soft constraints and returns unweighted structures from the exact continuous model. Weighted samples from the rounded model can be requested instead. Use :func:set_random_seed() to reproduce samples within a fixed software configuration; seeding NumPy's global RNG does not affect this method.

Parameters

`number` : int Number of structures to sample.

`weights` : bool, default=False If False, return unweighted structures sampled from the exact model. If True, return rounded-model samples and their log-weight corrections. The weighted path is faster when rounding corrections are present.

Returns

list of str or list of (str, float) With `weights=False`, a list of dot-bracket structures. With `weights=True`, each element contains a structure and its unnormalized log-weight correction

`log(w) = -(E_exact - E_rounded) / (k_B T).`

For state mixtures, the log-weight also includes the component-specific exact/rounded normalization correction. This correction is caused by rounding effective one-body and base-pair biases; state biases can contribute to an effective one-body bias when `reduce_state_space=True`.

When all effective component one-body and base-pair biases are multiples of 0.01 kcal/mol, every returned log-weight is zero. With `reduce_state_space=True`, differences between state biases become component one-body biases and may require rounding.

Method `suboptimal_coverage`

```
def suboptimal_coverage(
    self,
    delta
)
```

Return the fraction of the partition function represented by the suboptimal ensemble.

The suboptimal ensemble contains all structures whose exact energy is within delta kcal/mol of the minimum-free-energy structure.

Parameters

`delta : float` Maximum energy difference (kcal/mol) above the exact MFE.

Returns

float Fraction of the total partition function represented by the enumerated structures. The result lies between zero and one, apart from possible small numerical errors.

Raises

RuntimeError If no structure satisfies the hard constraints.

Method `suboptimal_structures`

```
def suboptimal_structures(
    self,
    delta
)
```

Enumerate suboptimal secondary structures.

Candidate structures are generated using ViennaRNA's rounded soft constraints, rescored with the exact continuous penalties, and returned sorted by exact energy.

For state mixtures, the MFE of every feasible state is calculated first. Each component is then enumerated with the local energy window required by the common, globally biased cutoff.

Parameters

`delta : float` Maximum energy difference (kcal/mol) above the exact MFE.

Returns

list of (str, float) List of (structure, energy) pairs sorted by increasing exact energy. The list can be passed directly to :meth:evaluate, including on another Molecule with the same sequence, to rescore the structures under a different set of biases.

Method `to_kwargs`

```
def to_kwargs(
    self
)
```

Return independent keyword arguments for an equivalent molecule.

The dictionary includes the sequence, all current biases, constraints, and resolved thermodynamic settings. Its arrays can be changed without changing this molecule. Pass it to `Molecule(**molecule.to_kwargs())` to construct a new model.

Method `total_free_energy`

```
def total_free_energy(
    self
)
```

Return the ensemble free energy.

Returns

float Ensemble free energy (kcal/mol) corresponding to the exact continuous pairing penalties.

Module `bussilab.wham`

Module containing a WHAM implementation.

See wham().

Changelog

- **v0.0.45**
  - Improved numerical stability for large bias energies.
- **v0.0.44**
  - T accepts one temperature per trajectory, as well as a scalar.
  - `normalize="log"` became the default, avoiding overflow or underflow when normalizing frame weights; boolean values remain supported.
- **v0.0.43**
  - Fixed `method="minimize"` with non-default traj_weight and frame_weight.
- **v0.0.42**
  - Temporarily used `method="substitute"` when traj_weight was supplied, avoiding incorrect results from the minimization method.
- **v0.0.41**
  - Changed the default solver to `method="minimize"` and the default convergence threshold to `1e-20`.
- **v0.0.28**
  - Added `method="minimize"`, minimize_opt, and WhamResult.nfev; substitution remained the default solver.
- **v0.0.27**
  - Added `normalize=False` and improved stability for large biases and initial logW/logZ estimates.
- **v0.0.2**
  - Introduced binless wham() with frame and trajectory weights, restart estimates, and WhamResult.

Functions

Function `wham`

```
def wham(
    bias,
    *,
    frame_weight=None,
    traj_weight=None,
    T: float = 1.0,
    maxiter: int = 1000,
    threshold: float = 1e-20,
    verbose: bool = False,
    logZ: numpy.ndarray | None = None,
    logW: numpy.ndarray | None = None,
    normalize: bool | str = 'log',
    method: str = 'minimize',
    minimize_opt: dict | None = None
)
```

Compute weights according to binless WHAM.

The main input for this calculation is in the 2D array bias. Element bias[i, j] should contain the energy of the i-th frame computed according to the j-th Hamiltonian. Trajectories should be concatenated first, so that the total number of frames should be equal to the number of frames of each trajectory multiplied by the number of trajectories. However, it is also possible to contatenate simulations of different lengths. It is crucial however to compute the potential according to each of the employed Hamiltonian on **all the frames**, not only on those simulated using that Hamiltonian.

Notice that by default weights are normalized. It is possible to override this behavior with normalize=False. However, starting with v0.0.40, this should not be necessary. The new implementation of normalization should be numerically stable in all cases.

Bugs

Up to version 0.042, method="minimize" does not work correctly when setting traj_weights. As a consequence, results produced with v0.0.41, where this method is the default, might be incorrect. In v0.0.42 this is temporarily fixed by reverting to method="substitute" when using traj_weights. In v0.0.43 this should be finally fixed: both methods should equally work in all cases, and the default "minimize" method should require less iterations.

Combining Trajectories Of Different Length

Let's imagine three frames obtained from three Hamiltonians. Let's assume that the energy of frame i in Hamiltonian j is given by bias[i, j] defined as

```
import numpy as np
bias = np.array([[1, 10, 7],
                 [2, 9, 6],
                 [3, 8, 5]])
```

We can compute the weights with the following command:

```
np.exp(wham.wham(bias).logW)
```

```
array([0.41728571, 0.39684866, 0.18586563])
```

We now notice that the second and third columns of this matrix are equal except for a rigid shift. They thus correspond to Hamiltonians that are equivalent. We should have been able to obtain the same result saying that these frames were coming from two simulations. If we only pass the first two columns however we obtain different weights

```
np.exp(wham.wham(bias[:, 0:2]).logW)
```

```
array([0.28224026, 0.43551948, 0.28224026])
```

In order to correcly analyze these frames we should pass the information that the second Hamiltonian was used for twice the time:

```
np.exp(wham.wham(bias[:, 0:2], traj_weight=(1, 2)).logW)
```

```
array([0.41728571, 0.39684866, 0.18586563])
```

Notice that now the weights are identical to those computed in the first example.

Trusting More A Trajectory Than Another

When you concatenate trajectories, you might explicitly want to trust more a trajectory than another. For instance, two trajectories might have been accumulated with a different stride, and the reliability of each frame of the one with smaller stride should be lower. We thus would like to assign a weight to each frame that accounts for its reliability.

Consider the following command:

```
import numpy as np
np.exp(wham.wham([[3, 5],
                  [4, 4],
                  [4, 4],
                  [4, 4],
                  [4, 4],
                  [4, 4],
                  [4, 4],
                  [4, 4],
                  [4, 4],
```

```
                  [4, 4],
                  [4, 4]]).logW)
```

that results in the following weights

```
array([0.06421006, 0.09357899, 0.09357899, 0.09357899, 0.09357899,
       0.09357899, 0.09357899, 0.09357899, 0.09357899, 0.09357899,
       0.09357899])
```

Notice that all the frames except for the first one are identical. An equivalent result would have been obtained using

```
import numpy as np
np.exp(wham.wham([[3, 5],
                  [4, 4]], frame_weight=(1, 10)).logW)

array([0.06421006, 0.93578994])
```

Clearly, the weight of the second frame in this example is equal to ten times the weights of the ten corresponding frames in the previous example.

Parameters

bias : np.ndarray An array with shape (nframes, ntraj) containing the bias potential applied to each frame according to each of the Hamiltonians.

frame_weight : np.ndarray, optional An array with nframes elements. These elements should contain the reliability weight of the frames. By default, these weights are set to one.

traj_weight : np.ndarray, optional An array with ntraj elements. These elements should contain the total weight of each of the Hamiltonians. Should be used when combining trajectories of different lengths.

T : float, optional The temperature of the system. This number is just used to divide the bias array in order to make it adimensional. In case replicas are simulated at different temperatures, it is possible to pass an array here, with ntraj elements.

maxiter : int, optional Maximum number of iterations in the minimization procedure.

threshold : float, optional Threshold for the minimization procedure.

verbose : bool, optional If True, print information as the minimization proceeds.

logZ : np.ndarray, optional Array with ntraj elements. Initial value for the logarithm of the partition functions. If not provided, it is computed from the bias. Providing an initial guess that is close to the converged value (e.g. as obtained from a calculation with a limited number of frames) can speed up significantly the convergence.

logW : np.ndarray, optional Array with nframes elements. Initial value for the logarithm of the weights. If not provided, they are computed from the bias. Providing an initial guess that is close to the converged value can speed up significantly the convergence. *If logW is provided, logZ is ignored.*

normalize : bool or str, optional By default, "log", which properly normalizes weights in all cases. normalize=True or False is enabled for backward compatibility.

method : str, optional If "substitute", solve self-consistent equations by substitution. If "minimize", use a minimization as in J Chem Phys 136, 144102 (2012). Prior to version 0.0.40, the default was "substitute". Starting with version 0.0.41, the default is "minimize".

minimize_opt : dict, optional If method=="minimize", this dict can be used to pass options to scipy.minimize. Notice that by default the minimization is performed using 'L-BFGS-B'.

Classes

Class `WhamResult`

```
class WhamResult(
    *,
    logW: numpy.ndarray,
    logZ: numpy.ndarray,
    nit: int,
    nfev: int,
```

```
        eps: float
    )
```

Result of a wham() calculation.

Ancestors (in MRO)

- bussilab.coretools.Result
- builtins.dict

Instance variables

Variable `eps` The final error in the iterative solution.

Variable `logW` numpy.ndarray containing the logarithm of the weight of the frames.

Variable `logZ` numpy.ndarray containing the logarithm of the partition function of each state.

Variable `nfev` The number of function evalutations (might differ from nit when using method='minimize').

Variable `nit` The number of performed iterations.

Generated by *pdoc* 0.11.6 (https://pdoc3.github.io).